

실습문제: 6장. 학습 관련 기술들(6.4, 6.5)

학번: 202404252 이름: 원종우

1. Overfitting

(1) 모델이 오버피팅되는 두 가지 경우는?

모델의 복잡도가 너무 높을 때: 파라미터 수가 많거나 모델이 너무 깊어서 학습데이터의 패턴뿐 아니라 노이즈까지 학습함.

학습 데이터가 너무 적거나 다양성이 부족할 때: 데이터가 충분히 다양한 분포를 대표하지 못해 모델이 **훈련 데이터에만 맞춰지는 현상**이 발생함.

(2) [그림 6-20] 그래프를 보고 오버피팅이라고 말할 수 있는 이유는?

훈련데이터에만 적응 해버렸기 때문이다

(3) [그림 6-21] 그래프가 오버피팅이 아니고 보는 이유는?

가중치 감소를 적용하여 훈련데이터와 테스트데이터의 차이를 줄였기 때문이다.

2. Weight decay

(1) 가중치 감소 무엇이고, 어떻게 구현하는가? 문장으로 설명하시오.

가중치 감소란 ****과적합을 방지하기 위해 큰 가중치에 패널티를 주는 정규화 기법(L2 정규화)****으로, 손실 함수에 가중치의 제곱합을 더해 **가중치가 과도하게 커지는 것을 억제하는 방법**이다.

구현은 손실 함수에 $\lambda * ||W||^2$ 항을 추가하거나, 옵티마이저 업데이트 단계에서

$W \leftarrow W - \eta(\partial L / \partial W + \lambda W)$ 처럼 **가중치에 비례한 감쇠 항을 함께 적용**하여 수행한다.

(2) [그림 6-21] 그래프가 오버피팅이 아니라고 보는 이유는? 정확도가 낮은 이유는 무엇일까?

오버피팅이 아닌 이유는 훈련 손실과 검증 손실이 서로 크게 벌어지지 않고 비슷한 흐름을 보여 과적합 패턴이 나타나지 않기 때문이다.

정확도가 낮은 이유는 모델의 표현력이 부족하거나 하이퍼파라미터 설정이 비효율적이라 데이터 패턴을 충분히 학습하지 못했기 때문이다.

(3) multi_layer_net.py 파일에서 Weight decay가 구현된 부분을 찾아 캡처하여 넣으시오.

```
def loss(self, x, t):  
    .....  
  
    y = self.predict(x)  
  
    weight_decay = 0  
    for idx in range(1, self.hidden_layer_num + 2):  
        W = self.params['W' + str(idx)]  
        weight_decay += 0.5 * self.weight_decay_lambda * np.sum(W ** 2)  
  
    return self.last_layer.forward(y, t) + weight_decay
```

3. dropout

(1) 드롭아웃을 하는 이유는? 드롭아웃을 어떻게 구현하는가?

드롭아웃은 과적합 방지를 위해 학습 시 확률 p 로 일부 뉴런을 무작위로 제거하고, 테스트 시 제거된 효과를 반영해 활성화 값을 스케일링하는 방식으로 구현된다.

(2) common/layers.py 파일에서 테스트(시험)시 드롭아웃의 출력 코드를 찾아 적으시오. 이 값을 곱하는 이유를 추측해 보시오.

```
class Dropout:
```

```
.....
```

```
    http://arxiv.org/abs/1207.0580
```

```
.....
```

```
    def __init__(self, dropout_ratio=0.5):
        self.dropout_ratio = dropout_ratio
        self.mask = None
```

```
    def forward(self, x, train_flg=True):
        if train_flg:
            self.mask = np.random.rand(*x.shape) > self.dropout_ratio
            return x * self.mask
        else:
            return x * (1.0 - self.dropout_ratio)
```

```
    def backward(self, dout):
        return dout * self.mask
```

학습 시 일부 뉴런이 무작위로 제거되므로, 평균 출력 값이 학습과 테스트에서 달라진다. 따라서 테스트 단계에서는 제거된 뉴런 비율을 고려해 **출력 값을 $(1 - \text{dropout_ratio})$ 만큼 스케일링**하여 학습과 동일한 출력 수준을 유지한다. 즉, **출력값의 크기를 맞춰 모델의 예측이 안정적이도록 하기** 위함이다.

(3) dropout_ratio = 0.1이면, Dropout 계층에서 학습시 대략 몇 %의 노드를 제외시키는가?
10%

4. 최적의 하이퍼파라미터 결정

(1) 하이퍼파라미터와 파라미터는 각각 무엇을 의미하는가?

파라미터(Parameter): 학습을 통해 **모델이 스스로 학습하는 값** (예: 가중치, 편향)

하이퍼파라미터(Hyperparameter): 학습 전에 **사용자가 설정하는 값** (예: 학습률, 은닉층 수, 배치 크기)

(2) 검증 데이터(Validation data)란?

- **모델 학습 중 성능을 평가하기** 위해 학습 데이터와 별도로 떼어놓은 데이터
- 과적합을 방지하고 **하이퍼파라미터를 조정**하는 데 사용됨

(3) 6.5.2를 참고하여 최적(최선)의 하이퍼파라미터를 결정(선택)하는 절차(4단계)를 설명하시오.

0단계: 하이퍼파라미터 값의 범위를 설정한다

1단계: 설정된 범위에서 하이퍼파라미터의 값을 무작위로 추출한다

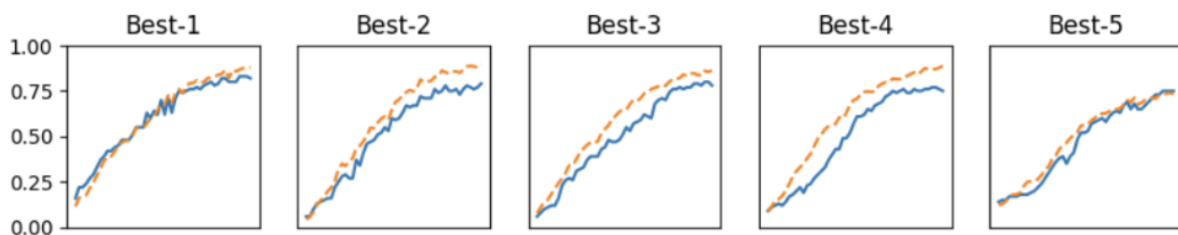
2단계: 1단계에서 샘플링한 하이퍼파라미터 값을 사용하여 학습하고, 검증 데이터로 정확도를 평가한다

3단계: 1단계와 2단계를 특정 횟수 반복하며, 그 정확도의 결과를 보고 하이퍼파라미터의 범위를 좁힌다.

※ 아래 (4)-(6)은 ch06/hyperparameter_optimization.py 코드를 보고 물음에 답하시오.

- (4) 6.5.3에서 두 하이퍼파라미터(학습률, 가중치감소 계수)를 최적화하였다. hyperparameter_optimization.py 파일을 각자 실행하여, p.226과 같이 Best-1부터 Best-5까지 결과를 캡처하여 붙여 넣으시오.

```
Best-1(val acc:0.82) | lr:0.006928418577235169, weight decay:7.47599743132482e-07
Best-2(val acc:0.79) | lr:0.0090866077857449, weight decay:2.8251660081378264e-07
Best-3(val acc:0.78) | lr:0.007792462762315259, weight decay:1.4925765354947992e-06
Best-4(val acc:0.75) | lr:0.008700605767881548, weight decay:1.7053909408929748e-07
Best-5(val acc:0.75) | lr:0.004016908697675404, weight decay:2.9154396199210318e-06
```



- (5) 위 (4) 이후 하이퍼파라미터의 범위를 좁혀야 한다. 범위를 어떻게 좁히겠는가? (힌트: 10 ** np.random.uniform(a, b) 대신 np.random.uniform(a, b)을 이용하면 더 쉽게 좁힐 수 있다)
초기 탐색에서 좋은 결과를 낸 하이퍼파라미터 값 주변으로 범위를 좁히고, 10 ** np.random.uniform(a, b) 대신 np.random.uniform(a, b)를 사용하여 선형 범위에서 세밀하게 탐색한다.”
- (6) 위 (5)에서 좁힌 범위에 대해 위 (5)를 수행하여 최적의 파라미터를 결정하시오. (힌트: 최적의 파라미터를 찾기 위해 몇 번 더 범위를 좁혀 보는 것이 좋다)
좁힌 범위(lr: 0.0005~0.002, weight_decay: 1e-7~5e-6)에서 탐색

※ 팀플시 이런 식의 하이퍼파라미터 최적화를 꼭 수행하시기 바랍니다.